

Trabalho Prático

Processamento de Texto com Haskell

Paradigmas de Linguagens
Wallace de Almeida Rodrigues

(Sem1, 2025)

Objetivo

O objetivo deste trabalho é desenvolver habilidades em programação funcional com Haskell por meio da construção de um programa que realiza análise de frequência de palavras em textos. O foco está na leitura e manipulação de arquivos, processamento de strings, uso de listas e conjuntos, e organização funcional do código.

Descrição da Tarefa

Você deverá implementar um programa em Haskell que leia um arquivo de texto e, para **cada parágrafo**, imprima uma lista de pares no formato:

(palavra, frequência)

onde:

- Cada parágrafo é definido como um bloco de texto separado por **uma ou mais linhas em branco**.
- Para cada parágrafo, a saída deve listar as palavras ordenadas **por frequência decrescente**.
- Palavras que pertencem a uma lista de *stopwords* (como artigos, conjunções e interjeições) **devem ser ignoradas** — ou seja, não devem ser contabilizadas nem exibidas.
- O programa deve **ignorar diferenças de maiúsculas/minúsculas** e **tratar palavras com ou sem acento como equivalentes** (ex: ação, Ação e ACAO devem ser consideradas a mesma palavra).
- Caracteres que não fazem parte de palavras (pontuação, símbolos, etc.) devem ser removidos.

Exemplo de Funcionamento

Considere o seguinte conteúdo em um arquivo `texto.txt`:

```
A ação é essencial.  Ação, reação, e interação!  
Outra linha.  Outra ação, outra reação.
```

E uma lista de *stopwords* que inclui: ["a", "e", "é"].
A saída esperada seria:

```
Parágrafo 1:  
("acao",3)  
("essencial",1)  
("reacao",1)  
("interacao",1)
```

```
Parágrafo 2:  
("outra",3)  
("linha",1)  
("acao",1)  
("reacao",1)
```

Requisitos Técnicos

- O programa deve ser implementado **exclusivamente em Haskell**.
- Você pode utilizar **apenas bibliotecas da base padrão**.
- A leitura do arquivo deve ser feita a partir do **primeiro argumento da linha de comando**.
- A lista de *stopwords* pode ser incluída diretamente no código como um conjunto (**Set**) de **String**.
- O tratamento de acentuação pode ser feito por substituição manual ou por bibliotecas que normalizem o texto.
- O código deve estar **organizado em funções puras**, com separação clara entre IO e lógica.

Entrega

- O trabalho deve ser entregue como:
 - Um arquivo `.hs` com o código-fonte;
 - Um arquivo `README.txt` com instruções para compilação e execução do programa;
 - Um **relatório** em PDF (de 2 ou 3 páginas), descrevendo como o trabalho foi realizado, as decisões de implementação, as bibliotecas e estruturas de dados utilizadas, o papel de cada função, os principais aprendizados e dificuldades.
- O trabalho pode ser feito em **grupo de até dois alunos**.
- Haverá uma **apresentação obrigatória e individual** do trabalho em data que será acertada com cada grupo.

Critérios de Avaliação

Critério	Peso
Correção da lógica funcional	3,0
Uso adequado de funções puras e imutabilidade	3,0
Legibilidade e organização do código	1,0
Tratamento de acentuação e capitalização	1,0
Documentação e clareza do relatório e README	2,0
Apresentação individual (! obrigatória !)	10,0

Aviso Importante

Este trabalho prático foi concebido como um **estudo dirigido**, visando o aprendizado ativo por meio da exploração e experimentação.

Sendo assim: são permitidas e incentivadas consultas a livros, materiais da internet (incluindo o uso de ferramentas como o ChatGPT), vídeos, fóruns, entre outros.

A única condição é que o(s) aluno(s) consiga(m) demonstrar o **entendimento do conteúdo** e o **aprendizado efetivo** durante a apresentação oral e por meio do relatório entregue. Trabalhos que apresentem evidências de simples cópia sem compreensão serão desconsiderados.